

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 3 – MEMÓRIA CACHE (AULA 20)

Política de Substituição/Despejo

- No caso de uma cache com mapeamento direto, quando uma entrada/linha está ocupada, porém não com o bloco necessário, é fácil saber que este bloco teria que ser substituído.
 - O TAG do endereço e TAG da entrada serão diferentes
- No caso de caches associativas (quando está cheia) ou caches associativas por conjunto (quando a linha está cheia) e bloco não se encontra na cache, algum bloco teria que ser retirado da cache para criar espaço para o novo bloco que precisará ser trazido da memória principal.
- A questão é qual bloco deve ser escolhido para ser removido?

Política de Substituição

- O mais simples em termos de custo, seria despejar a entrada mais velha, a entrada que entrou antes as outras
 - A cache pode ser implementada como um buffer circular, sem aumentar o tamanho de cada linha
- Alternativamente, a escolha poderia ser feita numa forma randômica
 - Surpreendentemente, funciona muito bem na prática
- A melhor escolha talvez seria despejar o bloco que ficou sem uso para o maior tempo, o *Least Recently Used* (LRU)
 - Mas como implementa esta política?
 - Uma maneira necessita de um contador para cada entrada. Cada vez a cache recebe uma solicitação, todos os contadores são incrementados e se tem um hit em uma linha, o contador dela é zerado.
 - O bloco LRU é aquele que tem a maior valor no contador
 - Porém, esta implementação fica muito cara por necessitar de um contador por linha, além de definir uma quantidade de bits suficiente para que o contador funcione bem.
 - Na prática, é usado uma forma aproximada que se chama *Pseudo LRU*
 - Dividindo a cache em grupos de entradas, o bloco mais velho do grupo LRU seria despojado.

Operações de escrita no cache

- Caches funcionam muito bem quando o processador precisa ler dados da memória
- Mas quando é necessário escrever?
 - O objetivo é alterar um valor em uma determinada posição (endereço) de memória.
 - Mas lembra que os acessos agora ao MP são feitos por blocos e
 - Tipicamente, só 15% das referências de memória são gravações
- A operação de escrita traz o seguinte dilema:
 - Devemos sempre atualizar os valores na MP para manter a consistência dos dados? [Técnica chamada *Write Through*]
 - Ou devemos fazer atualizações só na cache enquanto o bloco estiver lá? Quer dizer, só quando um bloco modificado é removido do cache a memória principal seria atualizada. [*Copy Back* ou *Write Back*]

Escrita: *Write Through*

- Sempre que o processador atualiza uma posição de memória, o dado é atualizado tanto na cache, quanto na MP
- Mantém a consistência dos dados da cache e da MP.
 - MP não tem dados com valores diferentes daqueles na cache.
- Isso é bom porque:
 - Simplifica substituição de bloco na cache;
 - Alguns dispositivos de E/S podem ler diretamente da MP, sem passar pela cache.
- Por outro lado, escritas feita pelo processador vão sempre ser lentas
 - Sempre precisam fazer acessos à MP.

Escrita: *Write Through* com *buffer*

- Caches que utilizam *write through* podem sofrer de escritas lentas, portanto, **os processadores normalmente incluem um buffer**, que enfileira escritas pendentes para a memória principal e permite que a CPU continue sua execução
- Buffers são comumente usados quando dois dispositivos são executados em velocidades diferentes.
 - Se um produtor gerar dados muito rapidamente para um consumidor manipular, os dados extras são armazenados em um buffer e o produtor pode continuar com outras tarefas sem esperar pelo consumidor.
 - Por outro lado, se o produtor desacelerar, o consumidor pode continuar rodando em velocidade máxima enquanto houver dados em excesso no buffer.
 - No caso de escrita, o produtor é a CPU e o consumidor é a memória principal.

Escrita: *Copy/Write Back*

- Nesta técnica, operações de escrita sempre só envolve a cache
- Apenas quando bloco é removido do cache, o conteúdo é atualizado na MP, se necessário.
 - Normalmente, há um bit em cada linha da cache, chamado *dirty bit*, que acusa se o bloco foi modificado e precisa ser atualizado na MP.
- Assim como as leituras, escritas também são tipicamente rápidas por acessar apenas a cache.
 - Se um único endereço é escrito com frequência, não vale a pena reescrever continuamente esses dados na memória principal.
 - Se várias palavras dentro do mesmo bloco de cache forem modificados, ainda vai acontecer apenas uma operação de escrita de memória no momento do *write-back*.

Escrita: *Copy/Write Back*

- Por outro lado:
 - Agora a cache e a memória contêm dados diferentes que quer dizer estão em um estado inconsistente!
 - Dispositivos de E/S que acessariam diretamente a memória precisam passar pela cache.
 - Isso pode sobrecarregá-la.
 - Há ainda a questão de como lidar com vários processadores/núcleos, cada um com sua própria cache.
 - Conhecida como o Problema de Coerência de Cache

Write around ou *Allocate-on-write*

- Ambas das duas políticas de escrita anteriores pode ser usado quando o bloco da palavra a ser escrita já está no cache
- Um segundo cenário seria quando o processador quer escrever uma palavra num bloco que não está na cache?
 - Com uma política de *write around* ou *write-no-allocate*, a operação de escrita vai diretamente para a principal memória sem afetar a cache.
 - Alternativamente, uma estratégia de *allocate-on-write* carregaria os dados recém-gravados na cache.

Protocolos de coerência de cache

- Protocolos de coerência de cache são essenciais em sistemas multiprocessadores para garantir que múltiplos caches mantenham uma visão consistente dos dados compartilhados.
 - Quando múltiplos processadores acessam e modificam o mesmo local de memória, podem surgir inconsistências, levando a um comportamento incorreto do programa.
 - Os protocolos de coerência de cache resolvem esse problema mantendo a uniformidade dos dados compartilhados em todos os caches.

Tipos de protocolos de coerência

□ *Snooping Protocols*

- Os protocolos de *snooping* necessita que cada cache monitorea as linhas de cache sendo acessadas. Quando um cache detecta uma gravação em um local de memória que ele contém, ele invalida ou atualiza sua cópia de acordo.
 - Esse método é mais rápido, mas menos escalável devido à necessidade de transmitir mensagens para todos os caches.

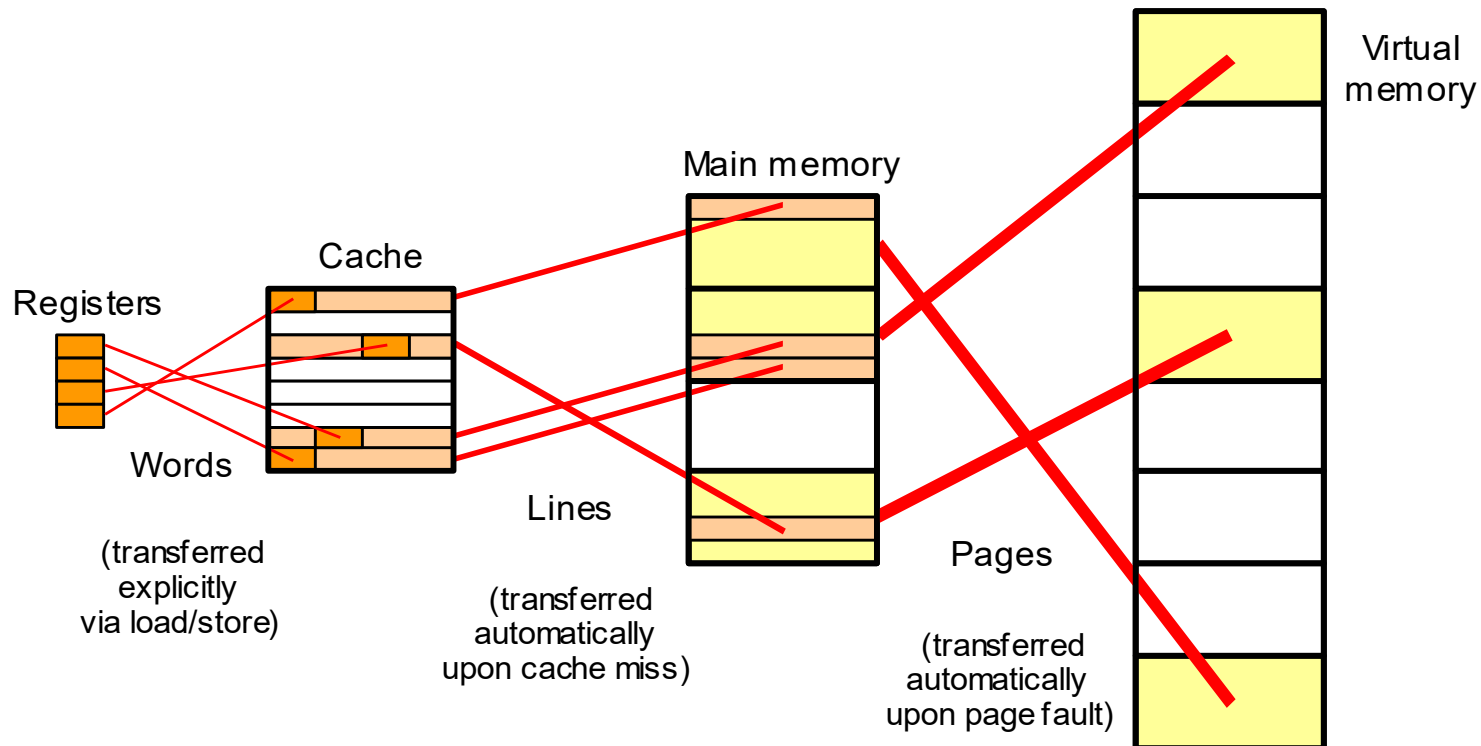
□ *Directory-based Protocols*

- Em protocolos baseados em diretório, o status de compartilhamento de um bloco de memória é mantido em um diretório central. Esse diretório rastreia quais caches possuem cópias do bloco e garante que quaisquer atualizações sejam propagadas para todos os caches relevantes.
 - Essa abordagem é escalável e eficiente para sistemas grandes.

Escrita: Um resumo

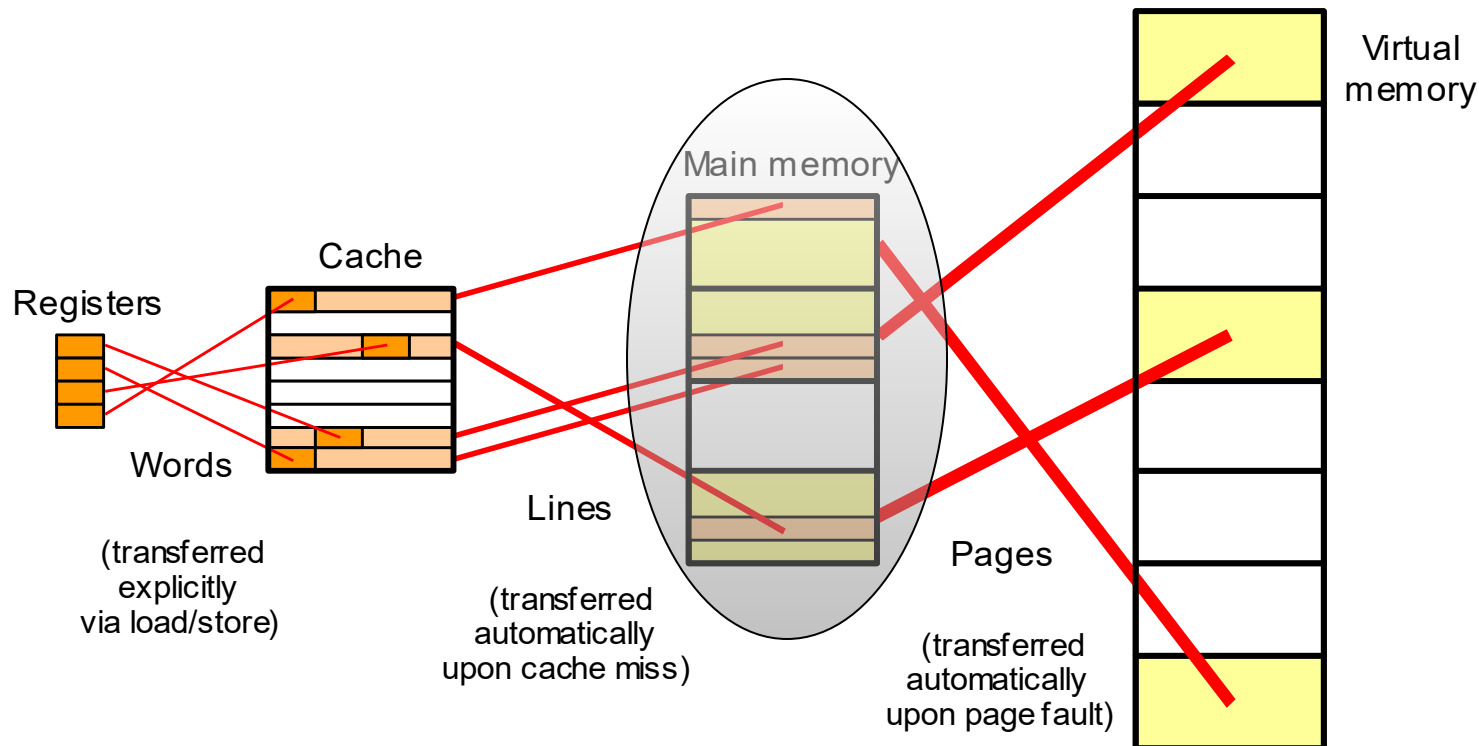
- Escrever em um cache apresenta alguns problemas interessantes
 - As políticas de *write-through* e *write-back* mantêm o cache consistente com memória principal de maneiras diferentes para acertos de escrita (*write hits*)
 - *Write-around* e *allocate-on-write* são duas estratégias para lidar com *write misses*, diferindo em se os dados atualizados são carregados no cache ou não
- O desempenho do sistema de memória depende do tempo de acerto do cache, taxa de falha e penalidade da falha, bem como o programa sendo executado.
 - Podemos usar esses custos para encontrar o tempo médio de acesso à memória.
 - $AMAT = \text{tempo de acerto} + (\text{taxa de falha} \times \text{penalidade de falha})$
- A organização de um sistema de memória afeta seu desempenho.
 - O tamanho do cache, o tamanho do bloco e a associatividade afetam a taxa de falha.
 - Podemos organizar a memória principal para ajudar a reduzir as penalidades por falhas.
 - Por exemplo, a memória intercalada oferece suporte a acessos de dados em pipeline.

Localidade na hierarquia de memória



Localidade na hierarquia de memória

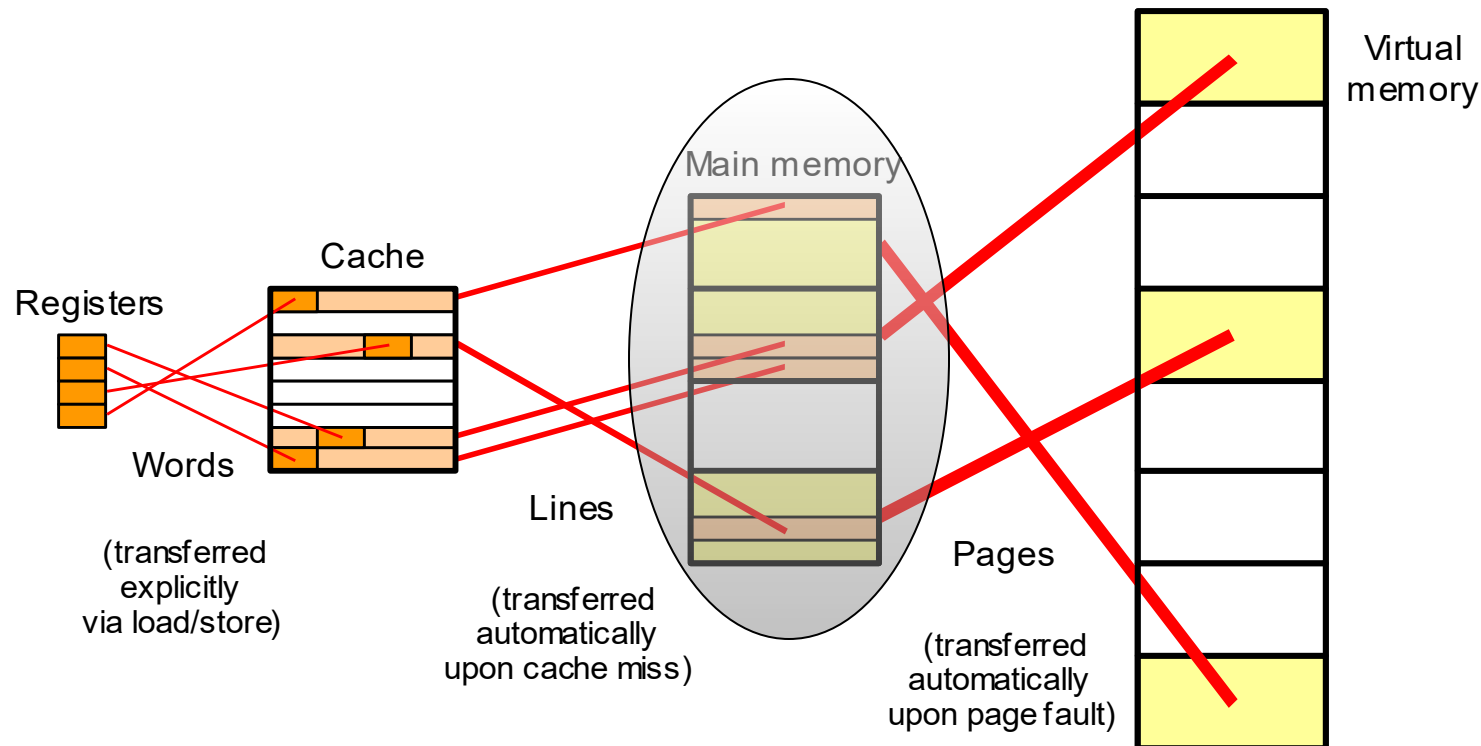
Memória Principal:
custo razoável, mas
é lento e pequena



Localidade na hierarquia de memória

Memory Cache:
fornece a ilusão
de alta velocidade

Memoria Principal:
custo razoável, mas
é lento e pequena

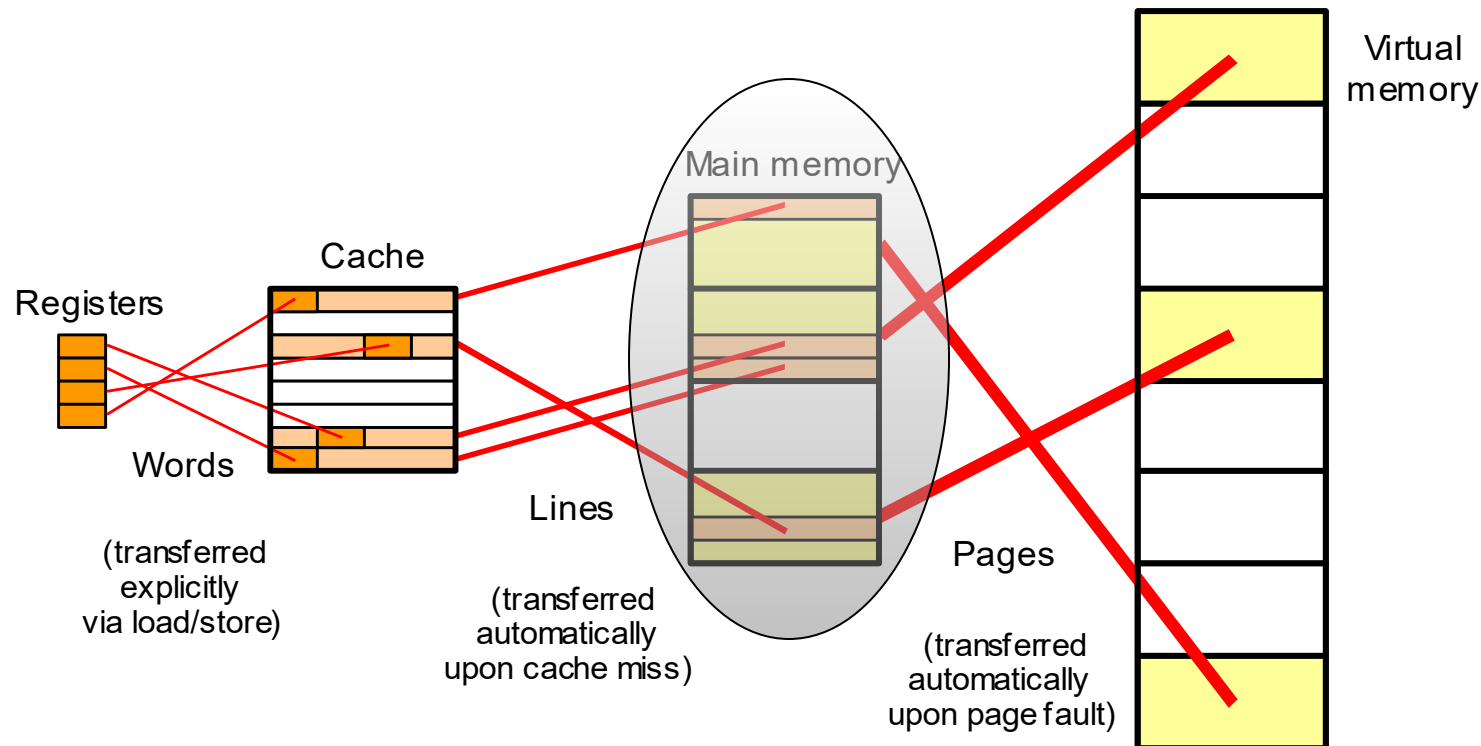


Localidade na hierarquia de memória

Memory Cache:
fornece a ilusão
de alta velocidade

Memoria Principal:
custo razoável, mas
é lento e pequena

Memory Virtual:
fornece a ilusão de um
tamanho muito grande

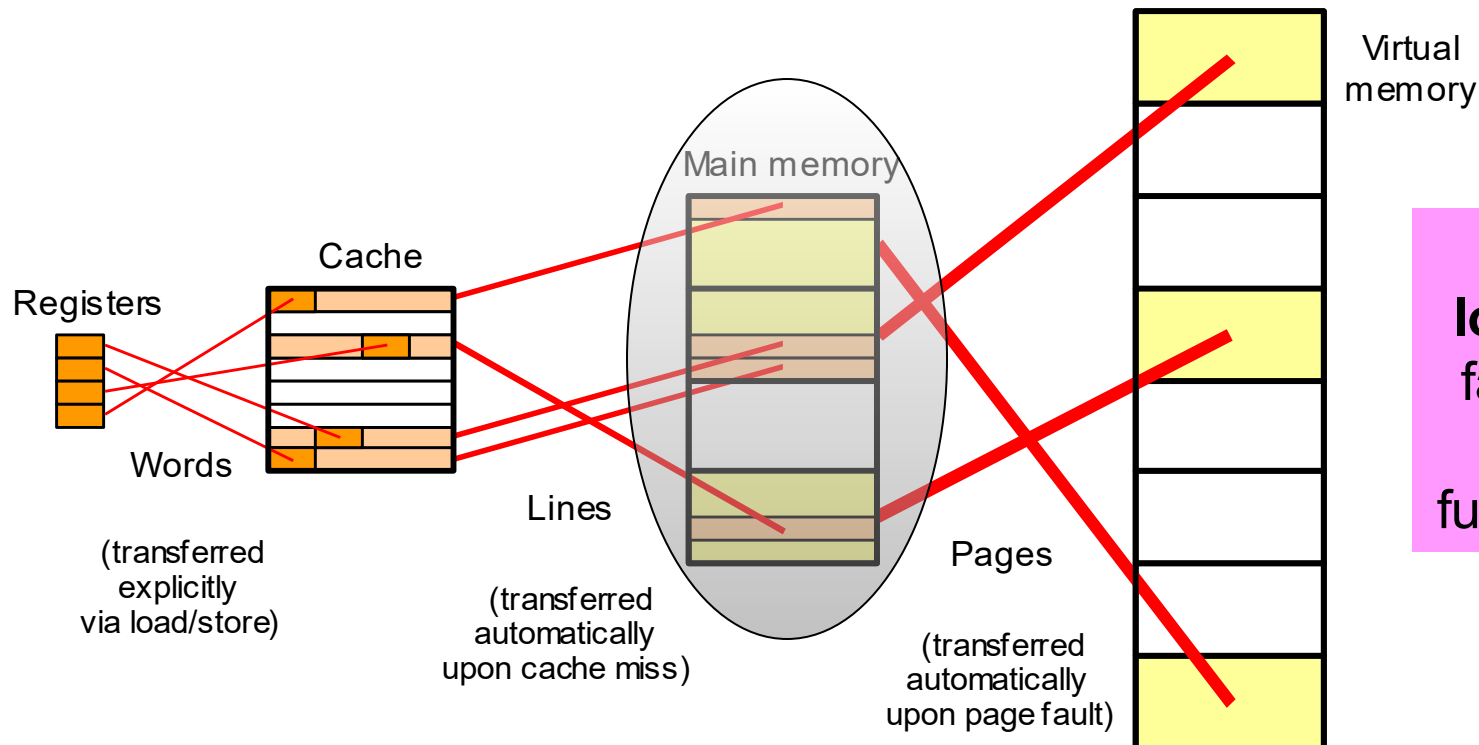


Localidade na hierarquia de memória

Memory Cache:
fornece a ilusão
de alta velocidade

Memoria Principal:
custo razoável, mas
é lento e pequena

Memory Virtual:
fornece a ilusão de um
tamanho muito grande



A
localidade
faz que as
ilusões
funcionarem